

Sandbox Reference Implementation vs Enterprise Production Target

```
+#####+ +#####+
| VERIFIED NOW (Sandbox Target) | | PRODUCTION TARGET (Enterprise) |
+#####+ +#####+
| • Single-node node:sqlite ACID engine | | • Multi-AZ PostgreSQL 16+ Cluster | |
| | In-process serialized transactions | | • Distributed resource coordination |
| | Local Ed25519 verification | | • Enterprise-managed key infrastructure|
| | Local SQLite SHA-256 audit ledger | | • Append-Only WORM Storage / QLDB |
| | Static Scoped Bearer Tokens | | • OIDC / OAuth 2.0 / BFF HttpOnly |
| | Synchronous in-memory outbox | | • Transactional Outbox + Worker Queue |
+#####+ +#####+
```

Production scaling requires migration from the verified single-node SQLite reference implementation to a distributed persistence architecture such as PostgreSQL, with appropriate transactional resource coordination and enterprise-managed key-management infrastructure.

 $\succ$

The exact distributed locking and key-management technologies are deployment decisions rather than requirements of the ACG authorization model.

1. **Persistence:** PostgreSQL with row-level locking (`SELECT . . . FOR UPDATE`) or distributed transaction managers.
2. **Resource Coordination:** Distributed transactional coordination (e.g., PostgreSQL advisory locks, Redis distributed locks, or etcd leases) selected based on deployment infrastructure.
3. **Key Infrastructure:** Enterprise-managed key management (e.g., AWS KMS, HashiCorp Vault, or cloud HSMs) for production signing and verification lifecycle.
4. **Session & Auth:** Enterprise OIDC / OAuth 2.0 / BFF HttpOnly cookie architecture for operator dashboard access.
5. **Durable Messaging:** Transactional outbox pattern for asynchronous fulfillment dispatch under network partitioning.